

**School of Computer Science and Artificial
Intelligence**

Lab Assignment # 15.5

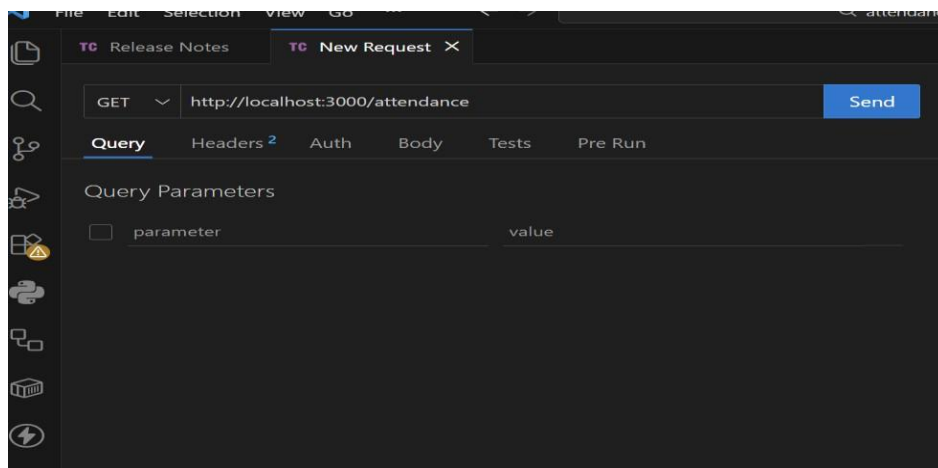
Program : B. Tech (CSE)
Specialization :AIML
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : VI
Academic Session : 2025-2026
Name of Student : R. ANIRUDH
Enrollment No. : 2303A52103
Batch No. : 33
Date :13/03/26

Task 1 – Student Attendance API

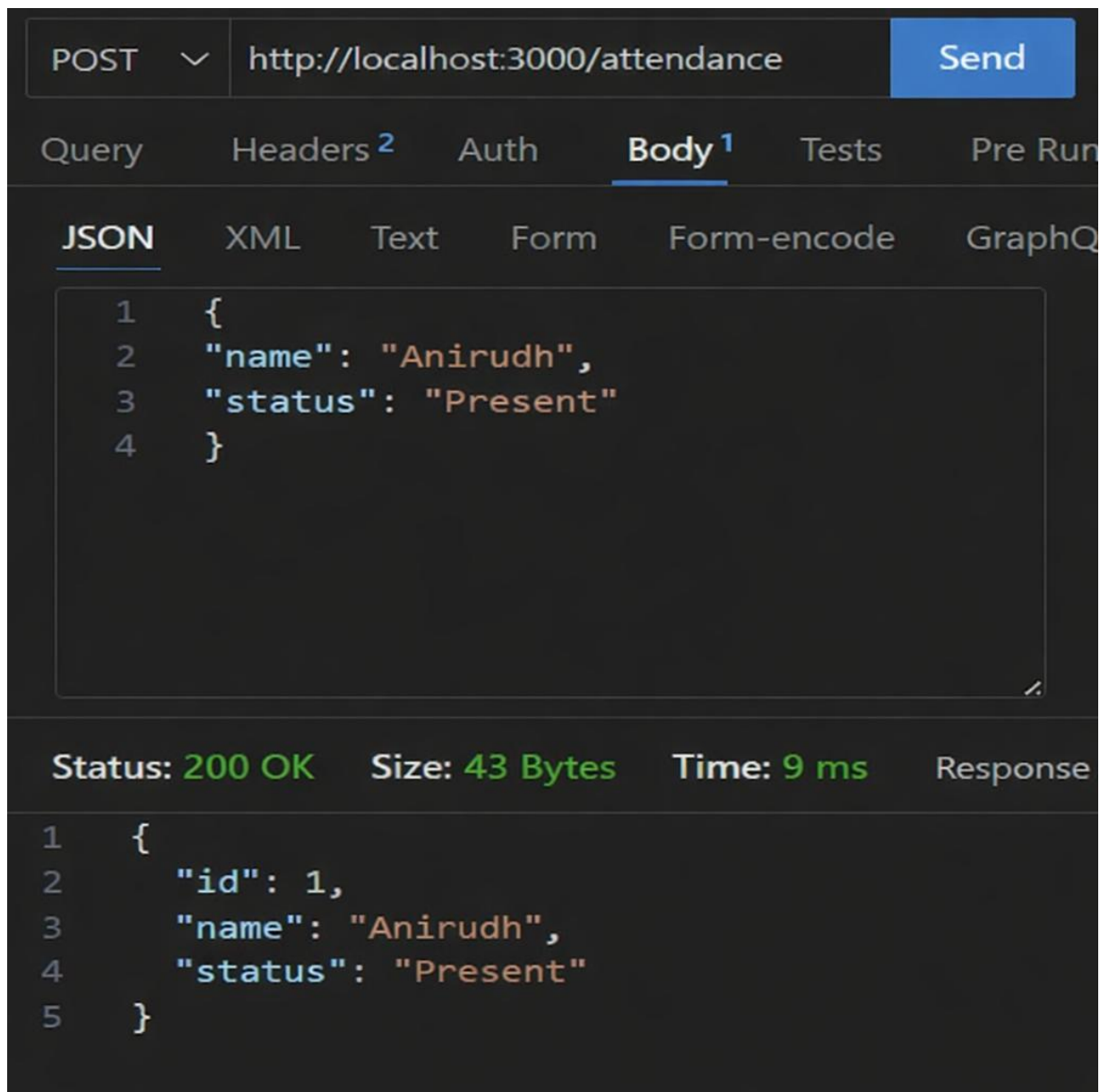
The screenshot shows a Visual Studio Code editor with a file named `server.js` open. The Explorer sidebar on the left shows a project structure with `node_modules`, `routes`, `package-lock.json`, `package.json`, and `server.js`. The `server.js` file contains the following JavaScript code:

```
1 const express = require("express");
2 const app = express();
3
4 app.use(express.json());
5
6 let attendance = [];
7
8 // GET attendance
9 app.get("/att let attendance: any[] {
10   res.json(attendance);
11 });
12
13 // POST attendance
14 app.post("/attendance", (req, res) => {
15   const record = {
16     id: attendance.length + 1,
17     name: req.body.name,
18     status: req.body.status
19   };
20
21   attendance.push(record);
22   res.json(record);
23 });
24
25 // UPDATE attendance
26 app.put("/attendance/:id", (req, res) => {
27   const id = req.params.id;
28
29   attendance = attendance.map(a =>
30     a.id == id ? { ...a, status: req.body.status } : a
31   );
32 });
```

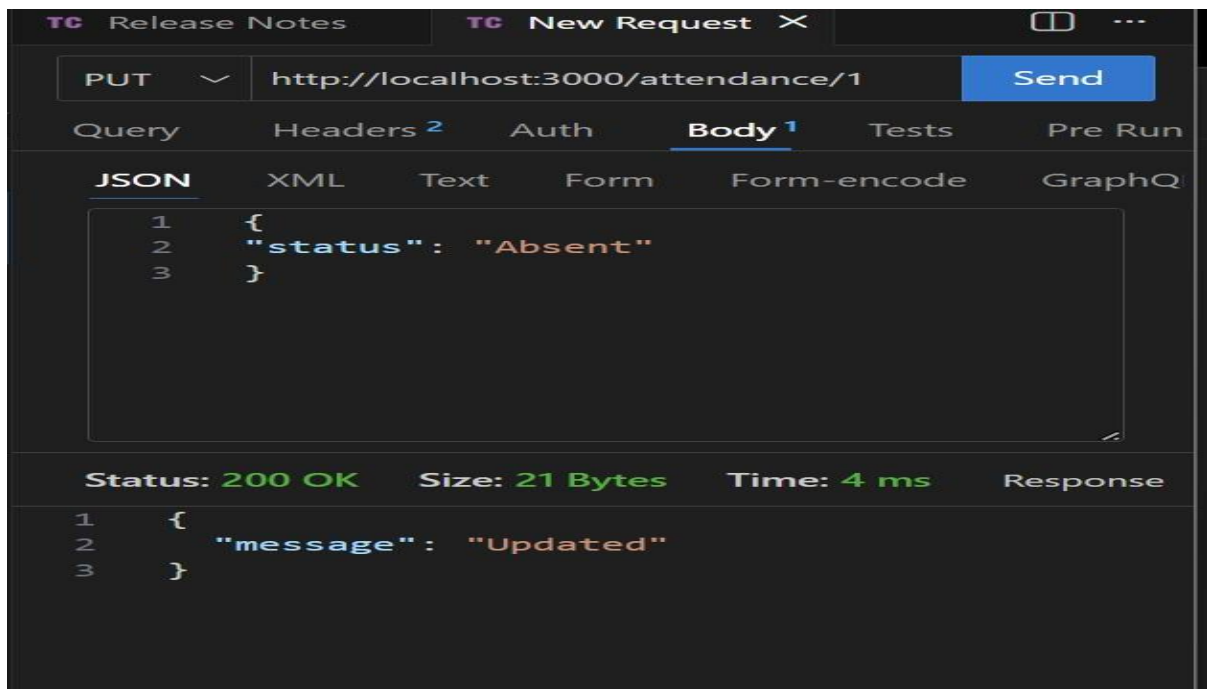
GET /attendance → View attendance records



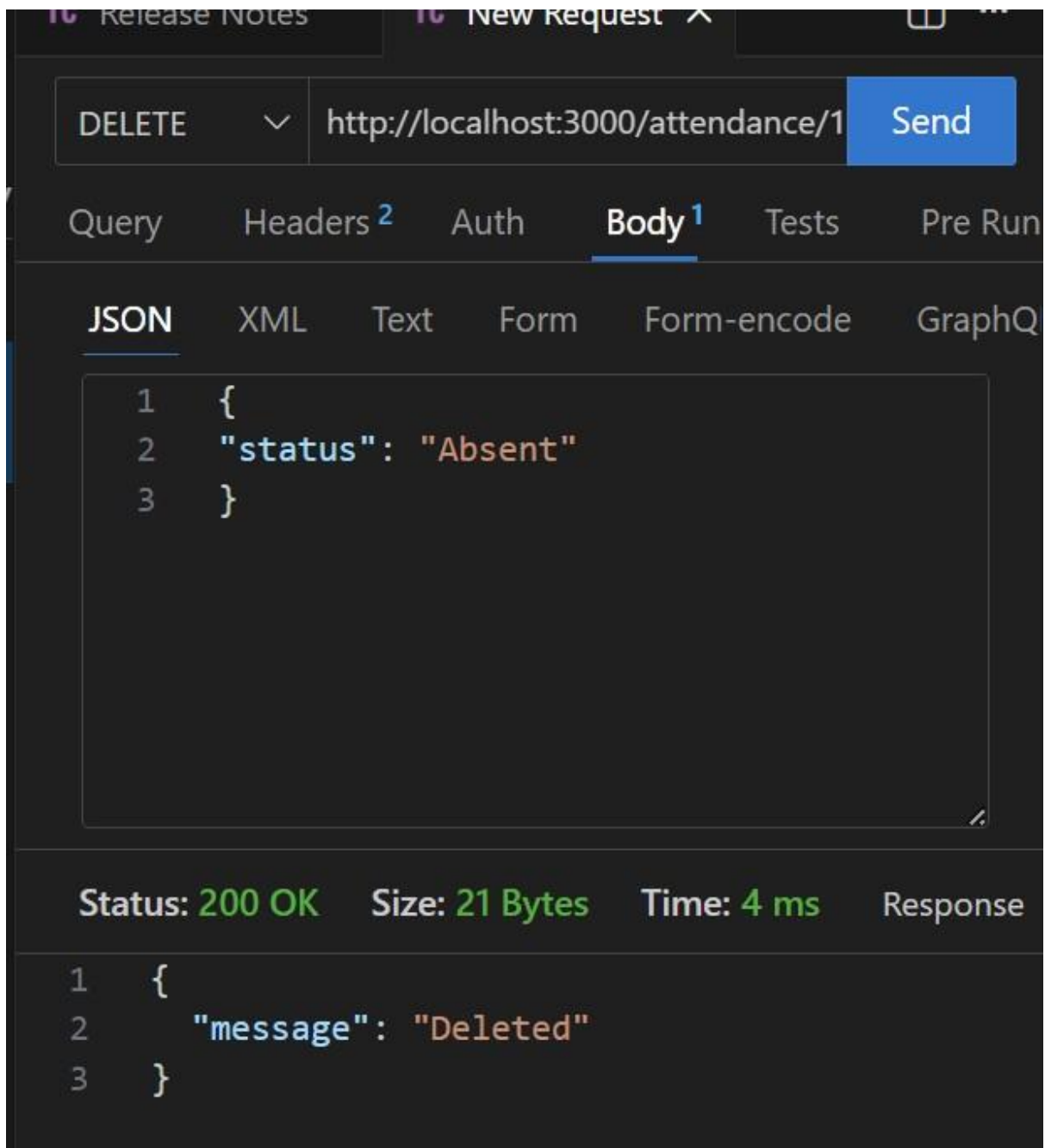
o POST /attendance → Mark attendance



o PUT /attendance/{id} → Update attendance record



o DELETE /attendance/{id} → Remove attendance entry



Task 2 – Inventory Management API

```
JS server.js X
JS server.js > ...
96
97 app.listen(3000, () => {
98   console.log("Server running on port 3000");
99 });

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
node + - - - - -

PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api\server.js:6
app.get("/inventory", (req, res) => {
^
ReferenceError: app is not defined
    at Object.<anonymous> (S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api\server.js:6:1)
    at Module._compile (node:internal/modules/cjs/loader:1761:14)
    at Object.<js> (node:internal/modules/cjs/loader:1893:10)
    at Module.load (node:internal/modules/cjs/loader:1481:32)
    at Module._load (node:internal/modules/cjs/loader:1300:12)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47

Node.js v24.13.0
PS S:\3 2 sem\AIAssisted_2026\Assignment15.5\attendance-api> node server.js
Server running on port 3000
```

GET /inventory → List all items

GET

Status: 200 OK Size: 2 Bytes Time: 9 ms

Query Headers² Auth Body¹ Tests Pre Run

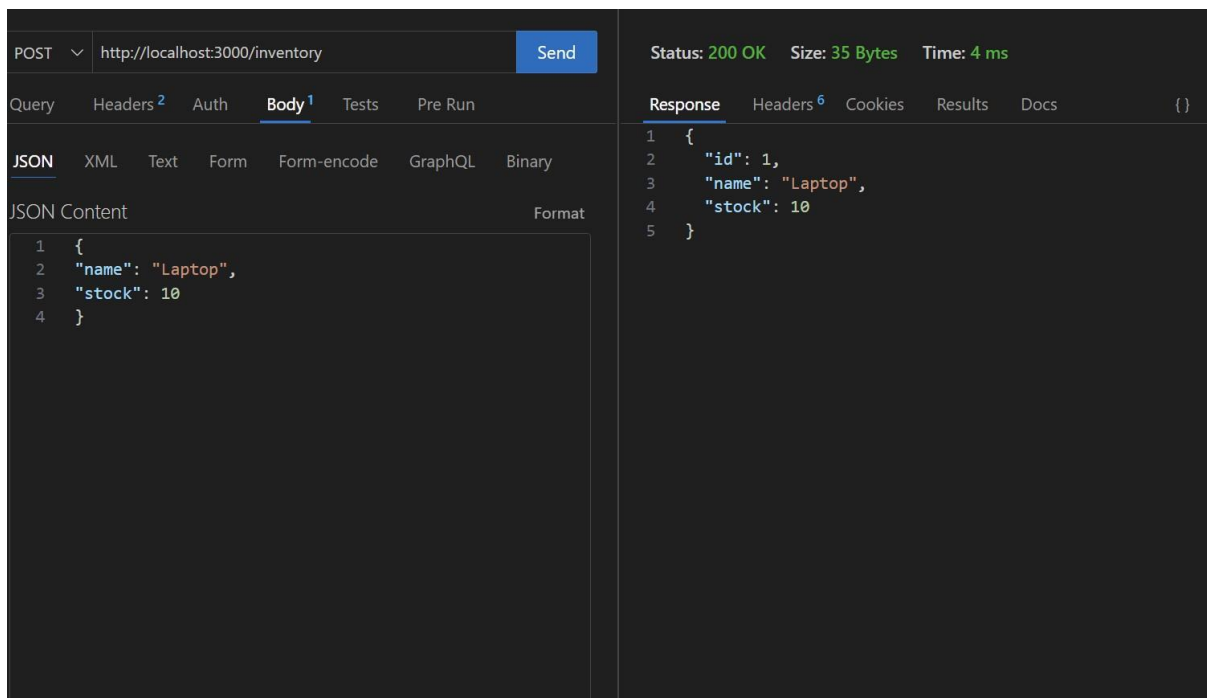
Response Headers⁶ Cookies Results Docs {}

Query Parameters

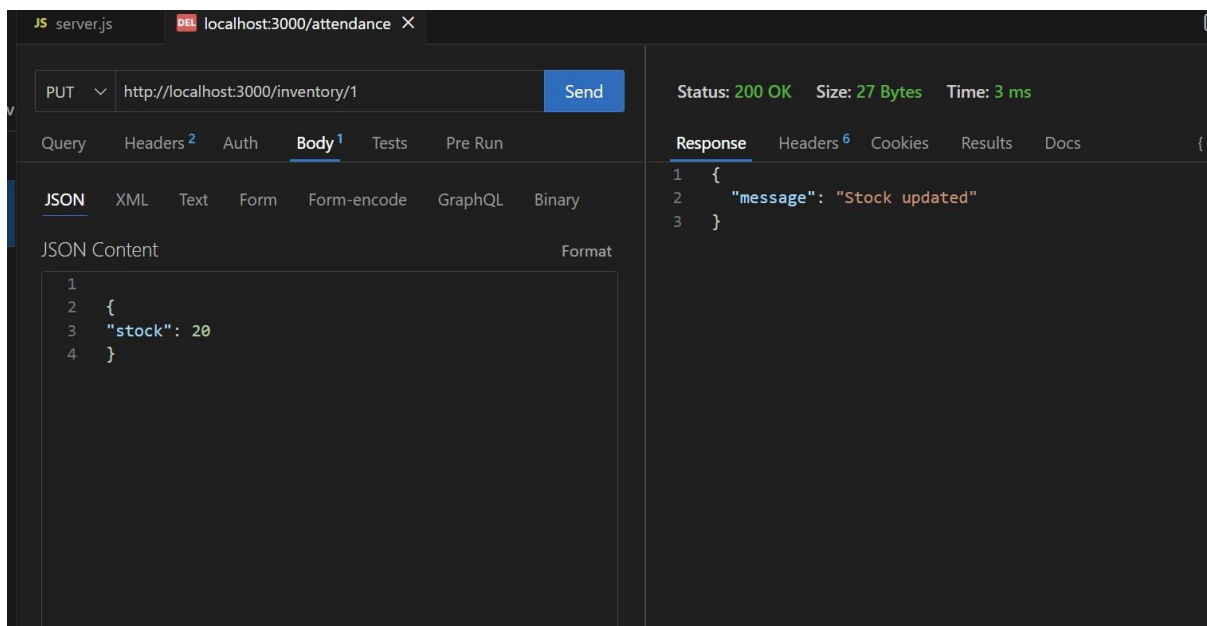
☐ parameter value

1 []

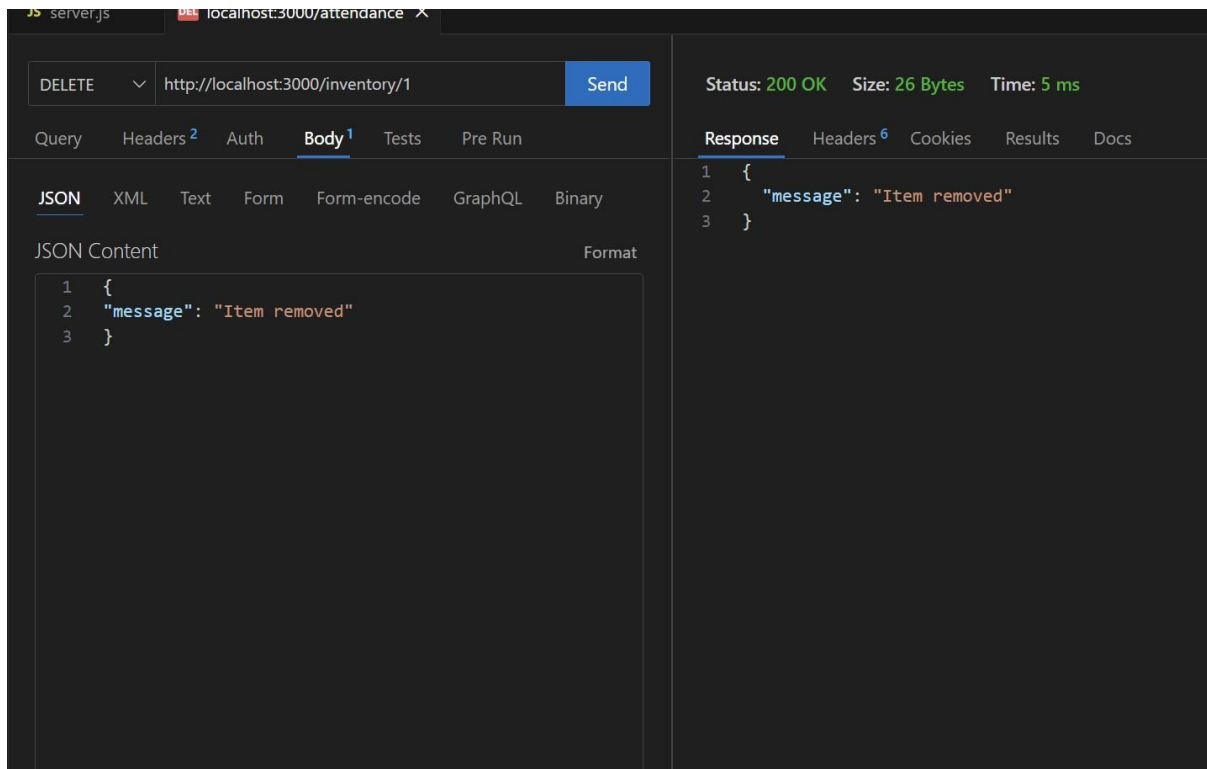
o POST /inventory → Add new item



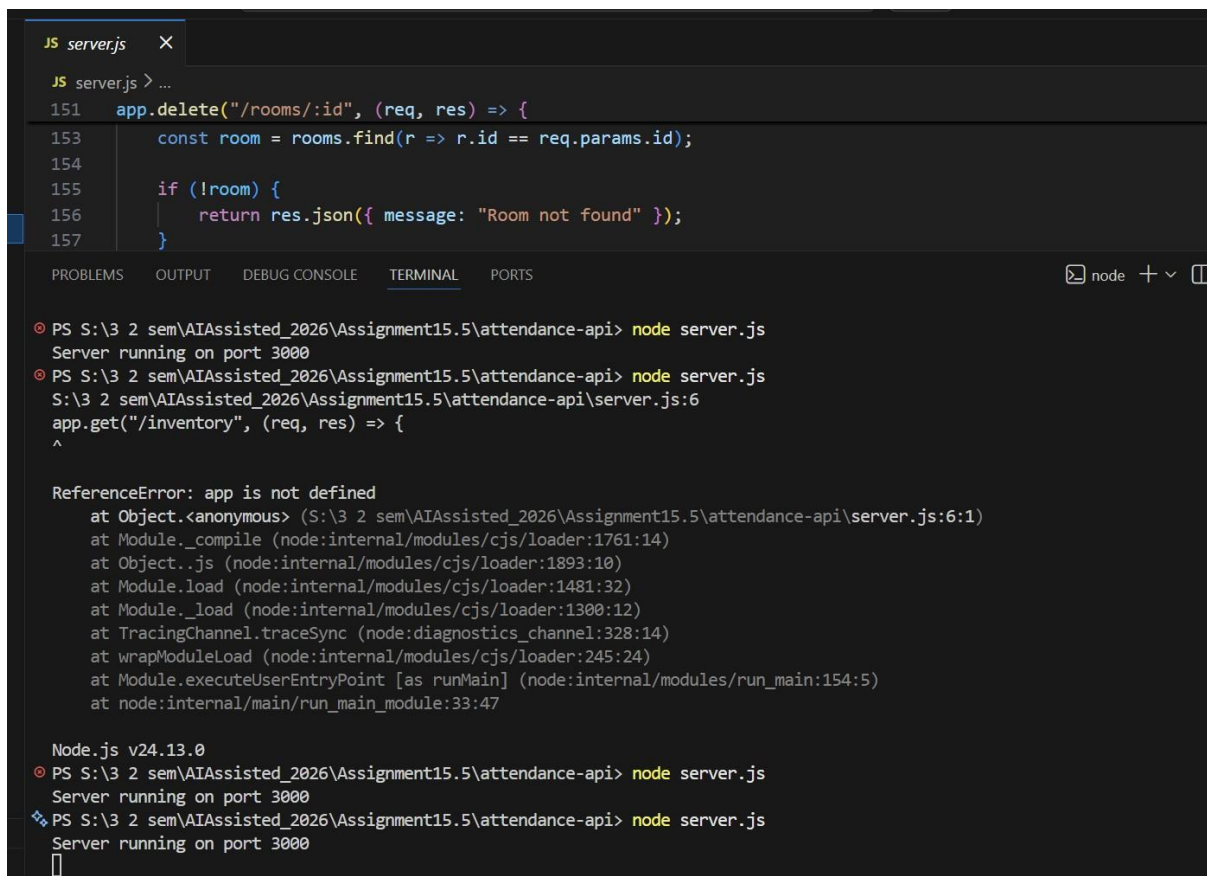
o PUT /inventory/{id} → Update stock quantity



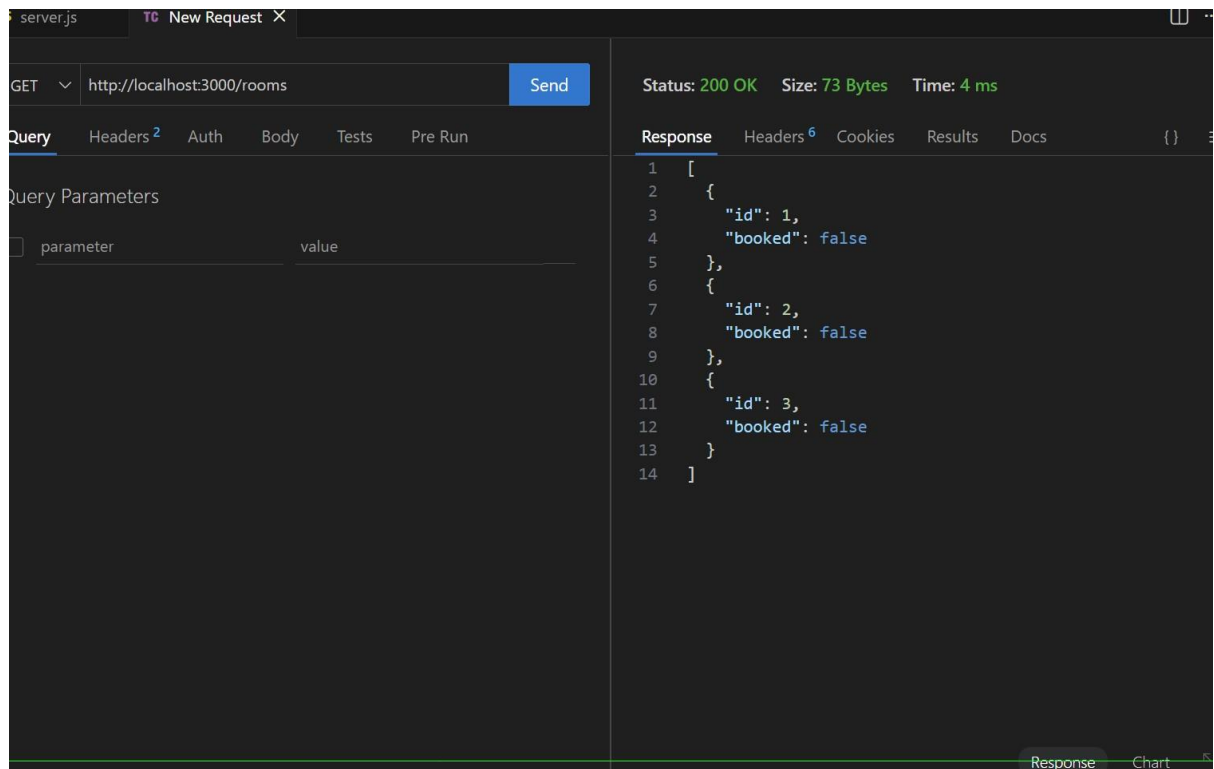
o DELETE /inventory/{id} → Remove an item



Task 3 – Hotel Room Booking API



o GET /rooms → View available rooms



server.js TC New Request X

GET http://localhost:3000/rooms Send

Status: 200 OK Size: 73 Bytes Time: 4 ms

Query Headers 2 Auth Body Tests Pre Run

Query Parameters

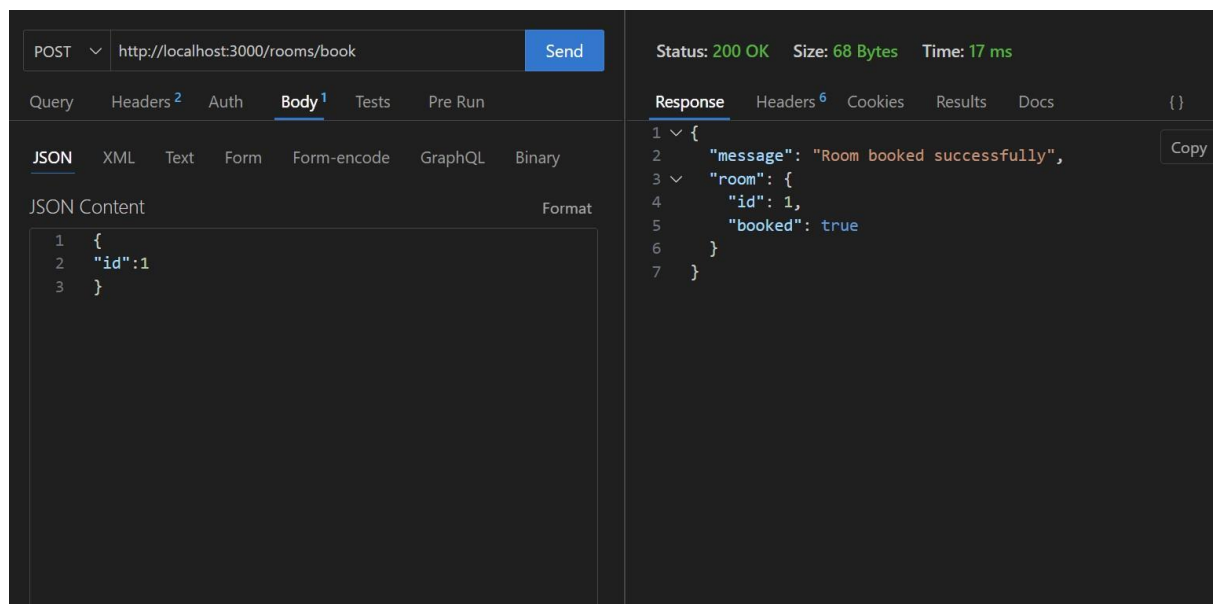
parameter	value
-----------	-------

Response Headers 6 Cookies Results Docs {}

```
1 [
2   {
3     "id": 1,
4     "booked": false
5   },
6   {
7     "id": 2,
8     "booked": false
9   },
10  {
11    "id": 3,
12    "booked": false
13  }
14 ]
```

Response Chart

o POST /rooms/book → Book a room



POST http://localhost:3000/rooms/book Send

Status: 200 OK Size: 68 Bytes Time: 17 ms

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id": 1
3 }
```

Response Headers 6 Cookies Results Docs {}

```
1 {
2   "message": "Room booked successfully",
3   "room": {
4     "id": 1,
5     "booked": true
6   }
7 }
```

Copy

o GET /rooms/{id} → View booking details

GET ⌵ http://localhost:3000/rooms/1 Send

Status: 200 OK Size: 22 Bytes Time: 4 ms

Query Headers² Auth Body¹ Tests Pre Run

Response Headers⁶ Cookies Results Docs

Query Parameters

☐	parameter	value
--------------------------	-----------	-------

```
1 {  
2   "id": 1,  
3   "booked": true  
4 }
```

o DELETE /rooms/{id} → Cancel booking

DELETE ⌵ http://localhost:3000/rooms/1 Send

Status: 200 OK Size: 31 Bytes Time: 2 ms

Query Headers² Auth Body¹ Tests Pre Run

Response Headers⁶ Cookies Results Docs

Query Parameters

☐	parameter	value
--------------------------	-----------	-------

```
1 {  
2   "message": "Booking cancelled"  
3 }
```

Task 4 – Online Voting API

The screenshot shows a VS Code editor with a file explorer on the left. The file explorer shows a project named 'ATTENDANCE-API' with files 'package-lock.json', 'package.json', and 'server.js'. The 'server.js' file is open in the editor. The code is as follows:

```
1 // Import express
2 const express = require("express");
3 const app = express();
4
5 app.use(express.json());
6
7 // Candidate data
8 let candidates = [
9   { id: 1, name: "Alice", votes: 0 },
10  { id: 2, name: "Bob", votes: 0 },
11  { id: 3, name: "Charlie", votes: 0 }
12 ];
13
14 // GET all candidates
15 app.get("/candidates", (req, res) => {
16   res.json(candidates);
17 });
18
19 // POST vote for candidate
20 app.post("/vote", (req, res) => {
21
22   const candidate = candidates.find(c => c.id == req.body.id);
23
24   if (!candidate) {
25     return res.json({ message: "Candidate not found" });
26   }
27
28   candidate.votes++;
29
30   res.json({
31     message: "Vote recorded"
```

o GET /candidates → List candidates

The screenshot shows a VS Code editor with a REST client window open. The request is a GET request to 'http://localhost:3000/candidates'. The response is a 200 OK status with a JSON array of candidate objects.

Request: GET http://localhost:3000/candidates

Response: Status: 200 OK Size: 103 Bytes Time: 8 ms

```
1 [
2   {
3     "id": 1,
4     "name": "Alice",
5     "votes": 0
6   },
7   {
8     "id": 2,
9     "name": "Bob",
10    "votes": 0
11  },
12  {
13    "id": 3,
14    "name": "Charlie",
15    "votes": 0
16  }
17 ]
```

o POST /vote → Cast a vote

POST http://localhost:3000/vote Send

Query Headers² Auth **Body¹** Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "id": 1
3 }
```

Status: 200 OK Size: 73 Bytes Time: 9 ms

Response Headers⁶ Cookies Results Docs

```
1 {
2   "message": "Vote recorded",
3   "candidate": {
4     "id": 1,
5     "name": "Alice",
6     "votes": 1
7   }
8 }
```

o GET /results → View voting results

GET http://localhost:3000/results Send

Query Headers² Auth Body¹ Tests Pre Run

Query Parameters

parameter	value
-----------	-------

Status: 200 OK Size: 158 Bytes Time: 3 ms

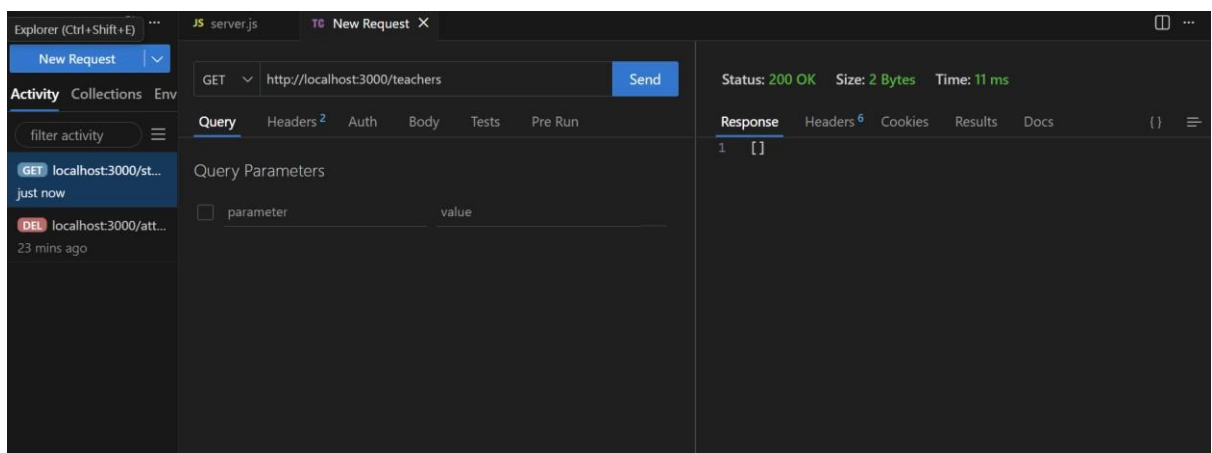
Response Headers⁶ Cookies Results Docs {}

```
1 {
2   "results": [
3     {
4       "id": 1,
5       "name": "Alice",
6       "votes": 1
7     },
8     {
9       "id": 2,
10      "name": "Bob",
11      "votes": 0
12     },
13     {
14       "id": 3,
15       "name": "Charlie",
16       "votes": 0
17     }
18   ],
19   "winner": {
20     "id": 1,
21     "name": "Alice",
22     "votes": 1
23   }
24 }
```

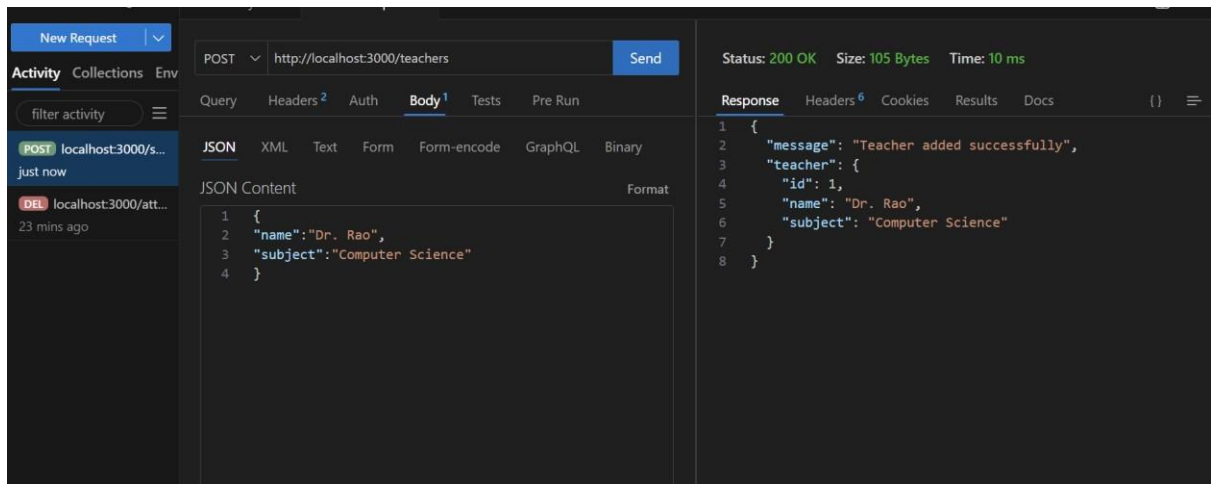
Task 5 – Teacher Management API

```
JS server.js x TC New Request
JS server.js > app.put("/teachers/:id") callback > message
37 app.put("/teachers/:id", (req, res) => {
45   teacher.name = req.body.name || teacher.name;
46   teacher.subject = req.body.subject || teacher.subject;
47
48   res.json({
49     message: "Teacher updated",
50     teacher: teacher
51   });
52 });
53
54 // DELETE /teachers/:id → Remove teacher
55 app.delete("/teachers/:id", (req, res) => {
56
57   const teacher = teachers.find(t => t.id == req.params.id);
58
59   if (!teacher) {
60     return res.status(404).json({ message: "Teacher not found" });
61   }
62
63   teachers = teachers.filter(t => t.id != req.params.id);
64
65   res.json({ message: "Teacher removed successfully" });
66 });
67
68 // Start server
69 app.listen(3000, () => {
70   console.log("Server running on port 3000");
71 });
```

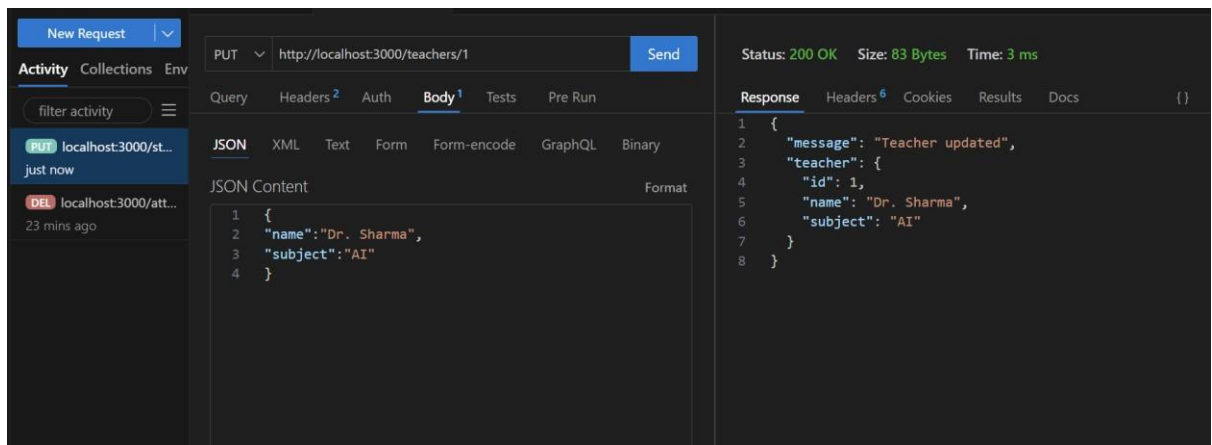
o GET /teachers → List all teachers



o POST /teachers → Add a new teacher



o PUT /teachers/{id} → Update teacher details



o DELETE /teachers/{id} → Remove a teacher

